

Week 4: An Introduction to Flutter

Dr M AOUE

Lecture notes

Objectives. By the end of this lecture you will know what a widget is and the difference between stateless and stateful widgets; you will be able to create a Flutter project and run it; and you will build a first Flutter app step by step with `MaterialApp`, `Scaffold`, `AppBar`, `Text`, a custom widget, `Column` and `Row`.

Every step of the lecture is a complete Flutter app. The files are in the `lib` folder of the project `flutter_examples`, inside `code/week04`. The listings below are taken directly from those files, and automated tests check that each app shows what is described here. See the guide *Running the programs in VS Code* to run them.

1 What is Flutter?

- Flutter is a collection of **widgets**, and **everything in Flutter is a widget**.
- A widget is a Dart class that represents a component of the user interface, such as an icon or a button.
- Every widget extends one of two classes: `StatelessWidget` or `StatefulWidget`.

Stateless widgets

A stateless widget extends `StatelessWidget`. Its state is fixed: it cannot change at run time. To show something different, a parent widget must build a new widget. Examples: `Text`, `Icon` and the buttons.

Stateful widgets

A stateful widget extends `StatefulWidget`. It can change its state (its contents) dynamically while the app runs, and redraws itself when `setState` is called. Examples: `Checkbox`, `Radio` and `TextField`.

The example below shows both kinds: `Greeting` is stateless (it only shows its name), `AcceptTerms` is stateful (it remembers whether the checkbox is ticked).

```

1 import 'package:flutter/material.dart';
2
3 // ----- Stateless: the content never changes by itself
4 // -----
5 class Greeting extends StatelessWidget {
6   final String name; // fixed input: set once, never modified
7   const Greeting({super.key, required this.name});
8
9   @override
10  Widget build(BuildContext context) {
11    return Text('Hello, $name');
12  }
13 }
```

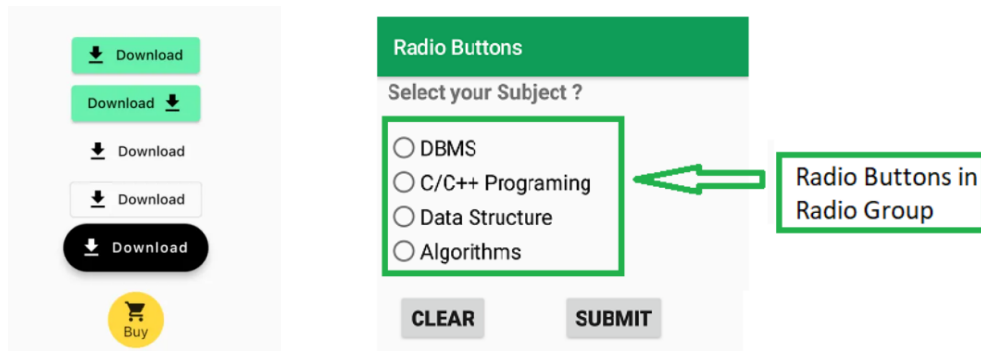


Figure 1: Stateless widgets: text, icons and buttons (left). Stateful widgets: radio buttons, which remember which one is selected (right).

```

13
14 // ----- Stateful: the content changes at run time -----
15 // The widget class is immutable; the changing data lives in its State object.
16 class AcceptTerms extends StatefulWidget {
17   const AcceptTerms({super.key});
18
19   @override
20   State<AcceptTerms> createState() => _AcceptTermsState();
21 }
22
23 class _AcceptTermsState extends State<AcceptTerms> {
24   bool _accepted = false; // the data that changes
25
26   @override
27   Widget build(BuildContext context) {
28     return Row(
29       mainAxisAlignment: MainAxisAlignment.min,
30       children: [
31         Checkbox(
32           value: _accepted,
33           // setState changes the data AND makes Flutter call build() again.
34           onChanged: (value) => setState(() => _accepted = value ?? false),
35         ),
36         Text(_accepted ? 'Accepted' : 'Not accepted'),
37       ],
38     );
39   }
40 }
41
42 // ----- Demo app (only used when this file is run) -----
43 void main() => runApp(const StatelessStatefulApp());
44
45 class StatelessStatefulApp extends StatelessWidget {
46   const StatelessStatefulApp({super.key});
47
48   @override
49   Widget build(BuildContext context) {
50     return const MaterialApp(
51       home: Scaffold(
52         body: Center(
53           child: Column(
54             mainAxisAlignment: MainAxisAlignment.min,
55             children: [Greeting(name: 'Rami'), AcceptTerms()],
56           ),
57         ),
58       ),
59     );

```

```

59     );
60   }
61 }

```

2 Creating a Flutter project

We create a simple “hello world” application.

With Android Studio

Start Android Studio and choose **File** → **New** → **New Flutter Project**. Check that the Flutter SDK path is correct and click **Next**. Type **week4** as the project name, choose the folder where the project is created, and click **Finish**.

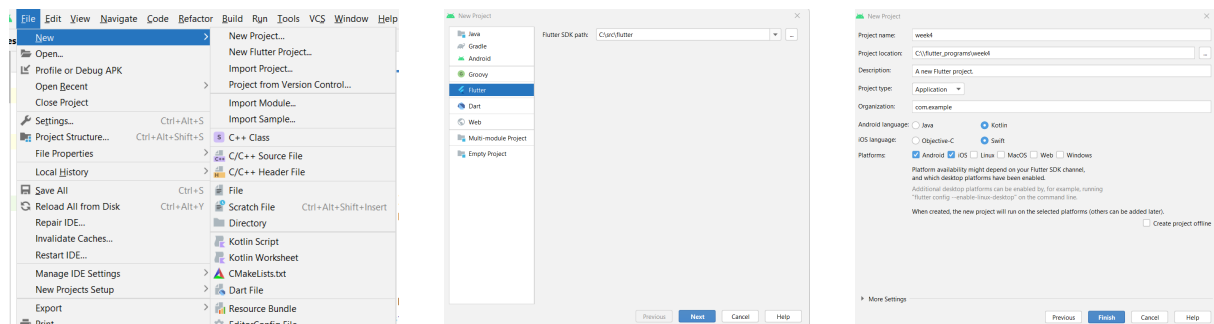


Figure 2: New Flutter Project in Android Studio: the menu, the SDK path, and the project name.

With VS Code

Open the Command Palette (**Ctrl+Shift+P**), run **Flutter: New Project**, choose **Application**, then choose a folder and a name. Or type in a terminal:

```
flutter create week4
```

In both cases Flutter creates a project that contains a sample application (a counter). The code of your app is in the file `lib/main.dart`.

3 Creating an emulator and running the app

To test the app on an Android phone you need a device: an emulator or a real phone.

- **Android Studio:** open the **Device Manager** and click **Create device**. When the device is created, click its **Launch** icon.
- **VS Code:** you can create the emulator in Android Studio’s Device Manager, or run the app on **Chrome** or on **Windows (desktop)**, which need no emulator. Choose the device at the bottom right of the VS Code window.

Then click the **Run** icon (in VS Code: **Run** above `main`, or **F5**). The sample application appears in the emulator: tap the **+** button to increase the counter.

4 The Hello World application

Now we create our own application. We replace all the code of `main.dart` with the code below, and then improve it step by step. Each step is one file of the project.

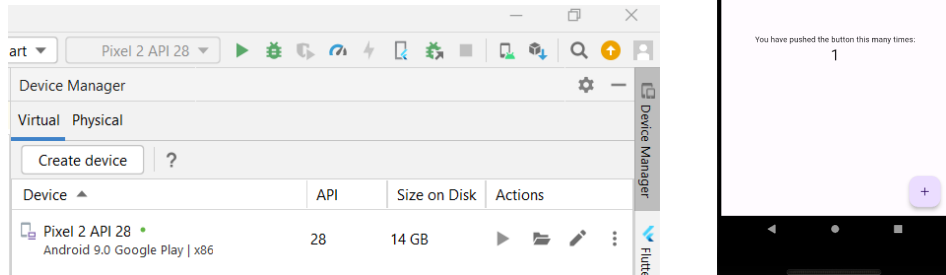


Figure 3: The Device Manager with an emulator (left), and the sample application running in it (right).

Step 1: the smallest app

```

1 import 'package:flutter/material.dart';
2
3 // Start the app: MyApp is the root widget. The lambda form is the same as
4 // void main() { runApp(const MyApp()); }
5 void main() => runApp(const MyApp());
6
7 class MyApp extends StatelessWidget {
8   // Older tutorials write: const MyApp({Key? key}) : super(key: key);
9   // "super.key" is the shorter form that current Flutter versions use.
10  const MyApp({super.key});
11
12  // build() describes what this widget looks like.
13  @override
14  Widget build(BuildContext context) {
15    // A widget that fills the screen with a color.
16    return Container(color: Colors.white);
17  }
18 }

```

- `import 'package:flutter/material.dart';` gives access to the widgets we need.
- Since this is Dart code, we define a `main` function. We use the lambda notation (week 2) to call `runApp`.
- `runApp` takes a widget and attaches it to the screen. By default the widget fills the whole screen. We pass it our `MyApp` widget.
- The constructor is called with `const`: `MyApp` is a stateless widget, so its state does not change at run time, and `const` makes the app run faster.
- `MyApp` is a stateless widget: the contents of the screen will not change. In Android Studio, type `st` and choose `stless` to create this class.
- Its `build` method returns the widget to display. An empty `Container()` shows a black screen, so we give it a color, for example white.

Step 2: displaying text

To display text we use a `Text` widget. We must wrap it in a `MaterialApp`, and we wrap the text in a `Center` to put it in the middle of the screen.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       // "home" is the widget shown on the screen.
8       home: Center(
9         // "child" is the widget that Center places in the middle.
10        child: Text('Hello World'),
11      ),
12    );
13  }
14 }

```

- **MaterialApp** gives the app the Material Design look. It has optional fields; **home** is the widget shown on the screen.
- By default the text would be displayed in the top left corner: **Center** places its **child** in the middle.
- Careful: **MaterialApp** alone gives the text an ugly style: red letters with a yellow underline. It is a warning style that tells the developer the app has not defined a default text style.

Step 3: adding a Scaffold

The warning style disappears when the text is inside a **Scaffold**, which gives the page a better background and text color. We use the **body** field of the **Scaffold** for the main content.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         // "body" is the main content of the Scaffold.
9         body: Center(child: Text('Hello World')),
10      ),
11    );
12  }
13 }

```

Step 4: adding an AppBar

The **appBar** field of the **Scaffold** takes an **AppBar**. Setting **centerTitle** to **true** centers its title.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(
9           title: Text('Home Page'),
10          centerTitle: true, // put the title in the middle of the bar
11        ),
12        body: Center(child: Text('Hello World')),
13      ),
14    );
15  }
16 }

```

```

14     );
15   }
16 }

```

Step 5: changing the text style

The `style` field of `Text` takes a `TextStyle`: here a font size of 24 and the color deep purple.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(title: Text('Home Page'), centerTitle: true),
9         body: Center(
10          child: Text(
11            'Hello World',
12            style: TextStyle(fontSize: 24, color: Colors.deepPurple),
13          ),
14        ),
15      ),
16    );
17  }
18 }

```

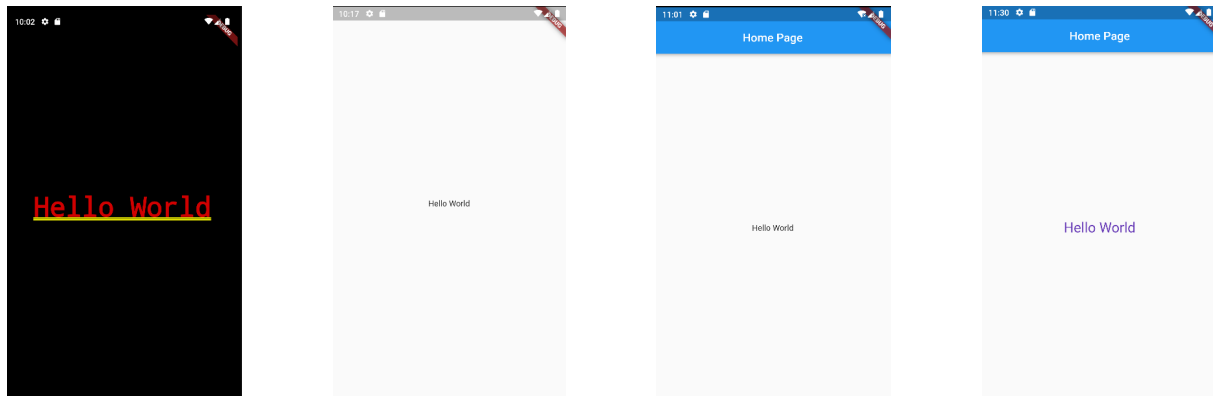


Figure 4: The result of steps 2 to 5, from left to right: `MaterialApp` alone, with a `Scaffold`, with an `AppBar`, and with a `TextStyle`.

Step 6: a custom widget

We can turn the `Text` into our own widget class, and reuse it whenever we need a similar text. In Android Studio, right-click the `Center` widget, choose **Refactor** and then **Extract Flutter Widget**; name it `MyTextWidget` and click **Refactor**. In VS Code, put the cursor on the widget, press `Ctrl+`, and choose **Extract Widget**.

Flutter replaces the body with `MyTextWidget()`. We then modify the widget to accept the text as a parameter: we add a field `text` and make it a required parameter of the constructor. Now the text is chosen when the widget is created.

```

1 // The custom widget: it shows any text in the purple style.
2 class MyTextWidget extends StatelessWidget {
3   final String text; // we add a field called text
4   // ...and make it a required parameter of the constructor

```

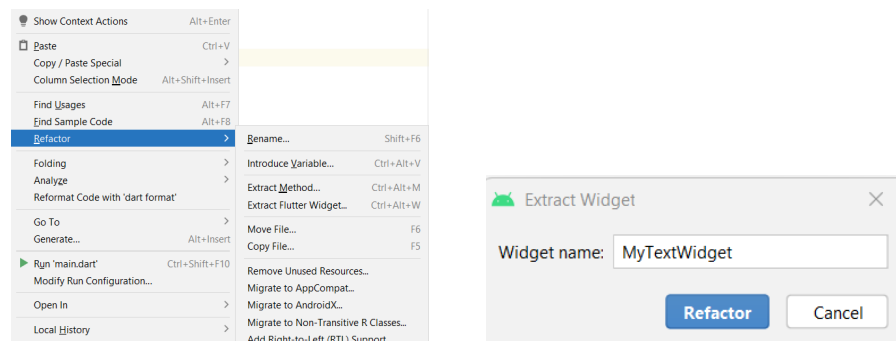


Figure 5: Extract Flutter Widget in Android Studio: the Refactor menu and the name dialog.

```

5  const MyTextWidget({required this.text, super.key});
6
7  @override
8  Widget build(BuildContext context) {
9    return Center(
10     child: Text(
11       text, // the field replaces the fixed 'Hello World'
12       style: TextStyle(fontSize: 24, color: Colors.deepPurple),
13     ),
14   );
15 }
16
17
18 class MyApp extends StatelessWidget {
19   const MyApp({super.key});
20
21   @override
22   Widget build(BuildContext context) {
23     return MaterialApp(
24       home: Scaffold(
25         appBar: AppBar(title: Text('Home Page'), centerTitle: true),
26         // now we can pass any text to the widget
27         body: MyTextWidget(text: 'Text 1'),
28       ),
29     );
30   }
31 }

```

Step 7: a Column

Column places widgets vertically. The widgets are given as a list in its children field. A SizedBox with a height of 16.0 adds empty vertical space between them.

```

1  class MyApp extends StatelessWidget {
2    const MyApp({super.key});
3
4    @override
5    Widget build(BuildContext context) {
6      return MaterialApp(
7        home: Scaffold(
8          appBar: AppBar(title: Text('Home Page'), centerTitle: true),
9          body: Column(
10             // The widgets of the column, from top to bottom.
11             children: <Widget>[
12               SizedBox(height: 16.0), // 16 logical pixels of empty space
13               MyTextWidget(text: 'Text 1'),
14               SizedBox(height: 16.0),

```

```

15         MyTextWidget(text: 'Text 2'),
16     ],
17 ),
18 ),
19 );
20 }
21 }

```

Step 8: a Row

Row aligns widgets horizontally. Here a Row with three texts is added to the Column, and `MainAxisAlignment.spaceEvenly` spreads them evenly.

```

1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return MaterialApp(
7       home: Scaffold(
8         appBar: AppBar(title: Text('Home Page'), centerTitle: true),
9         body: Column(
10          children: <Widget>[
11            SizedBox(height: 16.0),
12            MyTextWidget(text: 'Text 1'),
13            SizedBox(height: 16.0),
14            MyTextWidget(text: 'Text 2'),
15            SizedBox(height: 16.0),
16            // A Row nested inside the Column.
17            Row(
18              // Spread the three children evenly across the width.
19              mainAxisAlignment: MainAxisAlignment.spaceEvenly,
20              children: <Widget>[
21                MyTextWidget(text: 'Text 3'),
22                MyTextWidget(text: 'Text 4'),
23                MyTextWidget(text: 'Text 5'),
24              ],
25            ),
26          ],
27        ),
28      ),
29    );
30  }
31 }

```

5 Logical pixels

In the previous steps we used a `SizedBox` with a height of 16.0: it creates a vertical space of 16 **logical pixels**.

- A logical pixel is the scaling unit of Flutter.
- A logical pixel may be several physical pixels, depending on the screen of the device: it can be 10 physical pixels on one device and 20 on another. The ratio is the *device pixel ratio*.
- So a widget keeps the same size with respect to different screens.

The following app shows the size of your screen in both units, using `MediaQuery`:

```

1 import 'package:flutter/material.dart';
2
3 void main() => runApp(const LogicalPixelsApp());

```

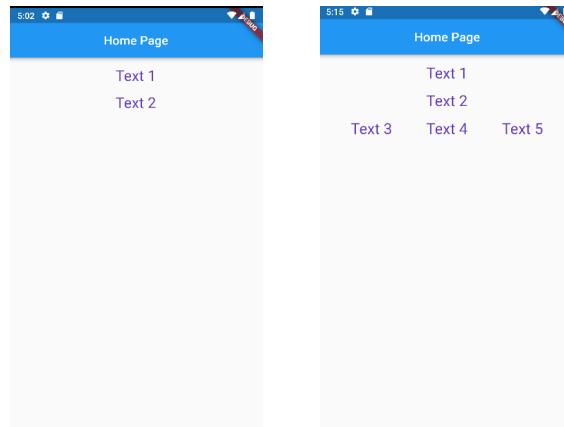


Figure 6: The result of step 7 (a Column, left) and step 8 (a Row in the Column, right).

```

4
5 class LogicalPixelsApp extends StatelessWidget {
6   const LogicalPixelsApp({super.key});
7
8   @override
9   Widget build(BuildContext context) {
10    return MaterialApp(
11      home: Scaffold(
12        appBar: AppBar(title: const Text('Logical pixels')),
13        body: const Center(child: ScreenFacts()),
14      ),
15    );
16  }
17 }
18
19 class ScreenFacts extends StatelessWidget {
20   const ScreenFacts({super.key});
21
22   @override
23   Widget build(BuildContext context) {
24     // Size of the screen (or window) in LOGICAL pixels.
25     final size = MediaQuery.sizeOf(context);
26     // How many physical pixels one logical pixel is.
27     final ratio = MediaQuery.devicePixelRatioOf(context);
28
29     return Column(
30       mainAxisAlignment: MainAxisAlignment.min,
31       children: [
32         Text('Logical size: ${size.width} x ${size.height}'),
33         Text('Device pixel ratio: $ratio'),
34         Text('Physical size: ${size.width * ratio} x ${size.height * ratio}'),
35         // The 16 of SizedBox(height: 16.0) is also logical pixels.
36         Text('16 logical pixels = ${16 * ratio} physical pixels'),
37       ],
38     );
39   }
40 }

```

6 Widgets at a glance

Widget	Purpose
Container	a box that can be given a color and a size
MaterialApp	the root of an app that uses Material Design (title, theme, home page)
Scaffold	the basic structure of a page: app bar, body, and more
AppBar	the bar at the top of a page, with a title
Center	centers its child widget
Text	shows a line of text; style takes a TextStyle
Column	places its children vertically
Row	places its children horizontally
SizedBox	an empty box, used for a fixed space between widgets

This document is maintained by Dr. Mohamad AOUDÉ
Last updated: 2026-09-21